Hessian-Trace Regularization for Looped Transformers: A Preregistered Single-Seed Pilot on Length Generalization

Libor "Burny" Burian

2026-11-01

Abstract

Looped (weight-tied, recurrent-depth) transformers can generalize to longer inputs by iterating a shared block more times, but this extrapolation varies strongly across random seeds. Flat-minima arguments suggest that lowering loss curvature might make extrapolating solutions more likely.

We preregistered a test of whether adding an exact Hutchinson estimate of the Hessian trace, $\lambda \cdot \mathrm{tr}(H)$, to the training loss of a looped transformer changes length generalization on binary addition. Models are trained on operands of 1–19 bits and tested on 20, 25, ..., 60 bits. This paper reports Stage 1: one penalized and one unpenalized model, with the same seed, initialization and data stream.

Stage 1 is in progress (finished arms: none).

We also found that PyTorch 2.15.0's native LayerNorm returns incorrect third derivatives. Any explicit Hessian-trace penalty through LayerNorm therefore needs a composite implementation. With one seed per arm, all conclusions are descriptive. The confirmatory, multi-seed design is preregistered and deferred.

1. Introduction and motivation

Looped transformers and length generalization.

- A looped transformer applies one block K times, with the input re-injected at each iteration (Dehghani et al. 2018; Yang et al. 2024; Fan et al. 2024).
- Because depth can grow at test time, such models can solve arithmetic and algorithmic problems longer than any seen in training (Fan et al. 2024).
- Kuo et al. 2026 show that this extrapolation is brittle. On binary addition, the across-run standard deviation of mean out-of-distribution (OOD) accuracy is 5.9–9.9 points for looped models trained with fixed or randomized loop counts (32 runs each; their Table 1), even among runs with similar in-distribution accuracy. Per length, the across-run SD is 8.7–12.2 points for looped models against 0.5–0.9 points for non-looped transformers (their Sec. 4.1).

The flat-minima hypothesis and its critics.

- The hypothesis: among minimizers of the training loss, those in flat regions generalize better (Hochreiter & Schmidhuber 1997; Keskar et al. 2017).
- Sharpness measures are among the better predictors of generalization in large studies (Jiang et al. 2020).
- In language models with equal pretraining loss, a lower Hessian trace correlates with better downstream accuracy (Liu et al. 2023; shown on synthetic PCFG-, HMM- and OPT-generated data).
- Counter-evidence and caveats:
  - Sharpness is not invariant to reparametrization (Dinh et al. 2017).
  - It often fails to track generalization in modern settings, including transformers, and sometimes correlates negatively with OOD error (Andriushchenko et al. 2023).
  - The maximum eigenvalue can change without any change in generalization (Kaur et al. 2023).

- For looped models in particular, Gong et al. 2025 conjecture, from empirical Hessian observations and theory in a simplified linear-attention setting, that looping biases training toward steep "V-shaped" valleys, and use this to explain looped models' better convergence.

Question. Does explicitly penalizing tr(H) during training change the length generalization of a looped transformer?

Our literature sweeps found no prior application of SAM, NSO or explicit Hessian-trace penalties to looped transformers.

2. Background: estimating and penalizing tr(H)

Hutchinson's estimator.

- $\mathrm{tr}(H) = E_v[v^T H v]$ for any v with $E[vv^T] = I$. With Rademacher v this is Hutchinson's estimator.
- Hv is computed without forming H, by double backpropagation, using the vector-Jacobian product of the gradient (reverse-over-reverse; PyTorch tutorial).
- Using $\lambda \cdot v^T H v$ as a training penalty requires its gradient $\nabla_\theta(v^T H(\theta) v)$, which is a third-order derivative of the loss.
- This penalty was used to regularize CNNs and LSTMs by Liu, Yu & Lin 2022 (SEHT).

Noise Stability Optimization (NSO) (Zhang, Li & Ju 2024).

- Update direction: $\frac{1}{2}[\nabla L(\theta+U) + \nabla L(\theta-U)]$, with $U \sim N(0, \sigma^2 I)$.
- Its expectation is $\nabla L + (\sigma^2/2)\nabla \mathrm{tr}(H) + O(\sigma^4)$. NSO is therefore a first-order, finite-difference estimator of the same objective, with $\lambda = \sigma^2/2$.
- Among Hessian-trace regularizers, it is the only one we found with transformer language-model results (GPT-2 chain-of-thought fine-tuning; a CLIP text encoder). SEHT's language-model results are for an LSTM.
- In LLM practice, flatness is otherwise pursued with SAM (Foret et al. 2021; Bahri et al. 2022) or with weight-noise and variational methods (Flat-LoRA; IVON).
- Stage 1 uses the exact Hutchinson penalty. NSO and SAM are Stage 2 arms.

3. Method

3.1 Task and data

Format. Binary addition in a variant of the full-output-prediction format of Fan et al. 2024:

- Input: `a + b >` followed by n+2 placeholders.
- Target: the (n+1)-bit sum, most significant bit first, then EOS.
- Loss is applied only on these n+2 answer positions.
- Fan et al.'s code instead starts the sum at the > position and supervises every trailing EOS position.

Training data.

- Operand lengths $n \in [1, 19]$.
- A linear curriculum raises the maximum length from 1 to 19 by the middle of training.

Test data.

- In-distribution (ID): n = 1–19. Out-of-distribution (OOD): $n \in \{20, 25, \ldots, 60\}$.
- 1000 problems per length, each length from its own seed.

3.2 Model

A weight-tied block of 3 pre-LayerNorm GPT-2 layers:

- d = 256, 8 heads, 4× MLP.
- No positional embeddings; causal mask; tied embeddings.
- Input injection: $h_t = \mathrm{Block}(h_{t-1} + e)$.
- 2.37M parameters.

Loop counts.

- Training: per-sample K_i ~ U{n_i−5, …, n_i+5}, clamped to [1, 20] (Kuo et al. 2026).
- Evaluation: every K = 1, …, n+5 is scored.

3.3 Training

Shared settings.

- AdamW (lr 1e-4, betas 0.9/0.999, weight decay 0.01), cosine decay to 0 over the whole run (as in Kuo et al. 2026; Fan et al. start the decay only after the curriculum ends).
- Batch 64, gradient clipping at 1.0, 30,000 steps.
- fp32 with TF32 matmuls.
- Both arms use seed 0. The data and loop-count stream comes from one generator, so the two models see identical batches from identical initial weights.

The penalized arm adds $\lambda \cdot v^T H_{sub} v$:

- one Rademacher probe per application;
- H_sub is the Hessian of the loss on the first 4 problems of the batch;
- applied every 8th step with weight 8λ, so the expected per-step penalty gradient is $\lambda \nabla \mathrm{tr}(H)$;
- $\lambda = 5 \times 10^{-5} = \sigma^2/2$ at NSO's central $\sigma = 0.01$. It was fixed in advance, not tuned.

The sub-batch size and the 8-step amortization were forced by the 8 GB GPU. They were recorded in a preregistration addendum before training (Appendix A).

3.4 Third-order autodiff correctness

The penalty's gradient was checked against a central finite difference of $v^T H v$ (step 10^{-5}, relative tolerance 10^{-5}) on a float64 copy of the model and on each operation separately.

Finding: PyTorch 2.15.0's native `layer_norm` gives incorrect third derivatives.

- All other operations in the model agree with finite differences: GELU, math scaled-dot-product attention, embedding, tied head, cross-entropy, and the per-sample read-out.
- Rademacher probes expose the error. A probe that only shifts LayerNorm's input by a constant across the normalized dimension (e.g. all ones) hides it, because LayerNorm is invariant to that shift and $v^T H v$ is then identically zero.
- We therefore build the penalty's graph with a composite LayerNorm (mean, variance, rsqrt). Elsewhere we keep the native kernel, whose first and second derivatives are correct.

3.5 Measurements

Primary metric: mean OOD exact match at the per-length best loop count ("oracle"). This adapts the oracle of Kuo et al., which picks the best K per example from {1, …, n+2}; ours picks one K per length from {1, …, n+5}, so their published numbers are not directly comparable.

Secondary metrics:

- Label-free maximum-confidence stopping (Fan et al.).
- K = n.
- ID accuracy.
- Curvature on a fixed batch of 128 training-distribution problems at K = n:
  - Hutchinson tr(H) with standard error, overall and per parameter group;
  - dominant eigenvalue by power iteration;
  - Fisher trace via sampled labels (the Gauss–Newton–Bartlett estimator of Sophia; a batched form of the estimator in Liu et al. 2023, App. B.1);
  - average-case adaptive sharpness (Andriushchenko et al. 2023);
  - weight norms.
- A loop-count study of tr(H) at fixed K.

Exact-match accuracy vs. operand length

Figure 1: Exact-match accuracy vs. operand length

Accuracy vs. loop count

Figure 2: Accuracy vs. loop count

Baselines that need no training: chance, and the untrained initialization.

3.6 Preregistered reading of a single-seed difference

With one seed per arm, seed variance cannot be estimated. Kuo et al. report an across-run SD of 5.9 points for mean OOD accuracy with randomized loop counts on addition (32 runs, their Table 1; measured with their per-example oracle, so it is an approximate reference here). With that SD, a difference between two single runs has an SD of about 8.3 points.

- A difference Δ in the primary metric is called "suggestive" only if $|\Delta| > 2 \cdot \sqrt{2} \cdot 5.9 \approx 16.7$ points; otherwise it is "inconclusive at n = 1 seed".
- Wilson intervals and a paired McNemar test cover test-set sampling only.
- **Manipulation check:** the penalized model's final tr(H) must be lower than the baseline's, with ±3 SE intervals that do not overlap.

4. Results

Pending.

Pending.

5. Discussion

6. Limitations

- **One seed per arm.** The across-run SD of looped models' mean OOD accuracy on addition is 5.9–9.9 points (Kuo et al. 2026, Table 1), so Stage 1 cannot separate an effect of the penalty from seed luck.
- **λ was not tuned.** It was fixed in advance from NSO's σ grid $\{0.008, 0.01, 0.012\}$, which NSO's paper reports for fine-tuning ResNet-34, not for a looped transformer trained from scratch.
- **The penalty is a high-variance estimator, further altered by the compute constraints.**
  - It uses one probe and 4 problems, applied every 8th step with an 8× weight.
  - On penalty steps, gradient clipping at 1.0 lets the penalty direction dominate the update.
  - The effect of curvature regularization is therefore confounded with injected gradient noise. Stage 2's random-direction control is designed to separate the two.
- **30k training steps,** against 100k in Fan et al. and Kuo et al.
- **The primary metric uses labels to pick the loop count per length.** Label-free metrics are reported alongside. It also differs from Kuo et al.'s per-example oracle, so it cannot be compared directly with their published numbers.
- **A single task** (binary addition).
- **Short ID lengths overlap with training problems,** which is unavoidable for small n.

Training dynamics

Figure 3: Training dynamics

Per-group Hessian traces

Figure 4: Per-group Hessian traces

Curvature vs. loop count

Figure 5: Curvature vs. loop count

7. Related work

- **Looped and recurrent-depth transformers:**
  - Universal Transformers (Dehghani et al. 2018);
  - looped transformers as programmable computers (Giannou et al. 2023);
  - looped transformers for in-context learning (Yang et al. 2024);
  - looped transformers for length generalization (Fan et al. 2024);
  - latent-thought reasoning (Saunshi et al. 2025);
  - recurrent-depth LLMs (Geiping et al. 2025; Ouro, Zhu et al. 2025);
  - randomized and learned stochastic stopping for stable extrapolation (Kuo et al. 2026).
- **Stability regularizers for looped and equilibrium models:**
  - Jacobian regularization of deep equilibrium models (Bai et al. 2021);
  - path independence (Anil et al. 2022);
  - spectral-radius regularization of looped language models (STARS, Yang et al. 2026).
  - These target the state-space Jacobian, not parameter-space curvature.
- **Curvature regularization:**
  - SAM (Foret et al. 2021), which with batch size 1 implicitly minimizes tr(H) while full-batch SAM minimizes the largest eigenvalue (Wen et al. 2023);
  - gradient-norm penalties (Zhao et al. 2022);
  - anticorrelated noise (Orvieto et al. 2022);
  - SEHT (Liu et al. 2022);
  - NSO (Zhang et al. 2024);
  - Hessian-based optimizers for LLMs (Sophia).

8. Conclusion

Appendix A: Preregistration and deviations

The preregistration (reports/preregistration.md) was committed before any code or results existed. Addendum 1 (before training) records:

- the benchmark-driven deviations (penalty sub-batch 16 → 4; penalty applied every 8 steps with an 8× weight);
- non-deterministic kernels;
- sequential runs;
- the composite LayerNorm in penalty graphs;
- the step budget S = 30,000.

Addendum 2 (before training) records a restart of the baseline run after its in-training curvature probe spilled out of GPU memory, a GPU memory cap, and a smaller curvature measurement (batch 512 → 128 problems; loop-count study with 10 probes and 10 power iterations instead of 20). Addendum 3 (a fact-check, before any Stage 1 evaluation) corrects how the task format, the oracle metric, the learning-rate schedule and several citations relate to their sources.

Appendix B: Per-length results

Pending.

Appendix C: Curvature details

Curvature measurements pending.

Loop-count study pending.

AI statement

This codebase and paper were written by Claude Code (Anthropic's Claude, model Opus 5.5) from an exact specification provided by the human author, who directed the research question, scope and constraints. Human cleanup and rewriting are in progress.